

Day 22 Notes on Generative AI - Retrieval Augmented Generation (RAG) with Gemini, FAISS and Streamlit

Introduction to RAG

Retrieval-Augmented Generation (RAG) is an AI architecture that combines information retrieval with large language models. Instead of relying only on the model's pre-trained knowledge, RAG retrieves relevant information from an external knowledge base and injects that information into the prompt before generating a response. This improves factual accuracy, reduces hallucinations, and allows AI systems to answer questions using domain-specific or up-to-date data.

Step 1: Installing Libraries

The notebook installs: google-generativeai – access Gemini models. faiss-cpu – vector similarity search engine. sentence-transformers – embedding generation. streamlit – web interface creation. pyngrok – exposing local applications to the internet. **Step 2: Import Libraries**

Key imports include Google Generative AI SDK, NumPy, FAISS, and SentenceTransformer. These libraries collectively handle embedding generation, vector indexing, retrieval, and response generation. **Step 3: Configure Gemini**

A Google API key is configured and a Gemini model is initialized. Gemini acts as the final reasoning and response-generation engine. The retrieved context is passed to Gemini so answers are grounded in actual documents. **Step 4: Create Knowledge Base**

A list of text documents acts as the external data source. Examples shown include FIFA World Cup information, Instagram trends, nutrition, cricket, and Gemini AI. In production systems this knowledge base may come from PDFs, databases, websites, research papers, or enterprise documents. **Step 5: Generate Embeddings**

SentenceTransformer (all-MiniLM-L6-v2) converts text into dense vector representations. Embeddings capture semantic meaning rather than simple keywords. Texts with similar meanings are mapped close together in vector space. Why embeddings matter: Enable semantic search. Capture context and intent. Allow matching even when wording differs. Provide numerical vectors suitable for machine learning. **Step 6: Create FAISS Index**

FAISS (Facebook AI Similarity Search) is used to store embeddings efficiently. The embedding dimension is obtained from the generated vectors. An IndexFlatL2 index is created and all document embeddings are added. Benefits of FAISS: Extremely fast similarity search. Scales to millions of vectors. Memory-efficient indexing. Widely used in production AI systems. **Step 7: Retrieve Relevant Documents**

The retrieve() function converts a user query into an embedding. FAISS searches the vector database and returns the nearest documents. The parameter top_k determines how many relevant documents are returned. Retrieval workflow: User enters query. Query converted into embedding. FAISS finds nearest vectors. Associated documents are returned. **Step 8: Build the RAG Function**

The rag_chat() function is the heart of the system. Retrieved documents are combined into a context block. A prompt template is created containing: Retrieved context. User question. Instructions for Gemini. Gemini receives both context and question, then generates a grounded response. Prompt structure: Context -> Retrieved Knowledge Question -> User Input Output -> AI Response Based on Context **Step 9: Test the RAG Pipeline**

A sample question about Machine Learning is passed to the system. Even if the exact answer is not present in the knowledge base, Gemini uses retrieved information and general reasoning to produce a useful response. This demonstrates how retrieval and generation work together.

End-to-End RAG Pipeline

User Query → Embedding Model → FAISS Search → Relevant Documents → Prompt Construction → Gemini → Final Answer **Advantages of RAG** Higher factual accuracy. Reduced hallucinations. Uses custom organizational knowledge. No need to retrain the LLM for every update. Supports real-time information retrieval. **Limitations** Quality depends on retrieved documents. Poor

embeddings can reduce retrieval quality. Large knowledge bases require optimization. Additional infrastructure compared to plain LLM usage. **Key Takeaways** RAG improves AI response accuracy by grounding answers in retrieved data. Streamlit simplifies AI application interface development. Vector embeddings enable semantic understanding of text. FAISS provides efficient similarity search and retrieval. Gemini enhances intelligent AI applications through powerful reasoning and generation. **Interview Questions**

1. What is RAG and why is it used?
2. How do embeddings differ from keywords?
3. What role does FAISS play in a RAG pipeline?
4. Why is semantic search important?
5. How does Gemini interact with retrieved context?
6. What are the benefits of external knowledge bases?
7. Explain the complete RAG workflow.
8. What challenges can occur during retrieval?